

Instrukcja implementacji wzorca projektowego Singleton

Język implementacji: Python

Cel instrukcji: Celem tej instrukcji jest wyjaśnienie zasady działania wzorca Singleton w taki sposób, aby druga osoba mogła zaimplementować go w języku Python bez korzystania z dodatkowych źródeł opisujących wzorzec.

1. Nazwa wzorca

Singleton, po polsku czasem nazywany pojedynczym obiektem. Jest to wzorzec kreacyjny, czyli związany ze sposobem tworzenia obiektów.

2. Problem

W programie czasami potrzebna jest klasa, z której powinien istnieć tylko jeden obiekt. Gdyby programista mógł tworzyć wiele obiektów tej klasy, mogłoby dojść do niespójności danych albo niepotrzebnego zużycia zasobów.

Przykładowy prosty temat implementacji: menedżer konfiguracji aplikacji. Program powinien mieć tylko jeden obiekt przechowujący ustawienia, np. tryb debugowania, nazwę aplikacji albo ścieżkę do pliku konfiguracyjnego.

Singleton można zastosować, gdy:

- w całej aplikacji powinien istnieć dokładnie jeden wspólny obiekt danej klasy,
- różne części programu powinny korzystać z tego samego stanu obiektu,
- tworzenie wielu kopii obiektu mogłoby powodować błędy lub chaos w danych.

3. Rozwiązanie

Rozwiązanie polega na takim zaprojektowaniu klasy, aby niezależnie od liczby prób utworzenia obiektu zwracana była zawsze ta sama instancja. W Pythonie można to zrobić przez nadpisanie metody `__new__()`.

Metoda `__new__()` odpowiada za faktyczne utworzenie obiektu, zanim zostanie wykonany konstruktor `__init__()`. Dzięki temu można sprawdzić, czy obiekt już istnieje. Jeśli nie istnieje, zostaje utworzony. Jeśli istnieje, klasa zwraca wcześniej utworzony obiekt.

Klasa Singleton powinna mieć:

- prywatne lub wewnętrzne pole klasowe, np. `_instance`, w którym przechowywana jest jedyna instancja,
- metodę `__new__()`, która kontroluje tworzenie obiektu,
- zwykłe metody obiektu, np. `set_value()`, `get_value()` albo `show_config()`, które pokazują, że wszystkie zmienne wskazują na ten sam obiekt.

4. Kroki implementacji w Pythonie

1. Utwórz klasę, np. ConfigManager.
2. Dodaj w klasie pole klasowe `_instance` ustawione początkowo na `None`.
3. Nadpisz metodę `__new__(cls)`.
4. W metodzie `__new__()` sprawdź, czy `cls._instance` ma wartość `None`.
5. Jeżeli `_instance` jest równe `None`, utwórz nowy obiekt za pomocą `super().__new__(cls)` i zapisz go w `cls._instance`.
6. Zwróć `cls._instance`.
7. Dodaj proste metody do zapisu i odczytu ustawień.
8. W programie głównym utwórz dwa obiekty tej klasy i pokaż, że wskazują na tę samą instancję.

5. Przykładowy kod pomocniczy

Poniższy kod pokazuje oczekiwany kierunek implementacji. Można go rozbudować, np. o słownik z ustawieniami aplikacji.

```
class ConfigManager:
    _instance = None

    def __new__(cls):
        # Tworzymy obiekt tylko przy pierwszym wywołaniu
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance.settings = {}
        return cls._instance

    def set_setting(self, key, value):
        self.settings[key] = value

    def get_setting(self, key):
        return self.settings.get(key)

config1 = ConfigManager()
config2 = ConfigManager()

config1.set_setting("debug", True)

print(config2.get_setting("debug")) # True
print(config1 is config2)           # True
```

6. Jak sprawdzić poprawność działania

Implementacja jest poprawna, jeśli po utworzeniu dwóch zmiennych, np. `config1` i `config2`, obie odnoszą się do tego samego obiektu. Można to sprawdzić operatorem `is`.

- `config1 is config2` powinno zwrócić `True`.
- Zmiana ustawienia przez `config1` powinna być widoczna po odczycie przez `config2`.
- Program nie powinien tworzyć osobnych, niezależnych konfiguracji.

7. Konsekwencje zastosowania wzorca

Zalety:

- zapewnia istnienie tylko jednego obiektu danej klasy,
- ułatwia dostęp do wspólnej konfiguracji lub wspólnego zasobu,
- zmniejsza ryzyko niespójności danych w różnych częściach programu,
- jest prosty do zaimplementowania w małych programach.

Wady:

- może wprowadzać globalny stan, który utrudnia testowanie programu,
- nadużywany Singleton może niepotrzebnie komplikować kod,
- w większych aplikacjach może utrudniać śledzenie, która część programu zmienia wspólny obiekt.

8. Proponowany prosty temat programu

Temat: Menedżer konfiguracji aplikacji w Pythonie.

Program powinien umożliwiać zapisanie kilku ustawień aplikacji w jednym wspólnym obiekcie. Następnie należy utworzyć dwie zmienne typu ConfigManager i pokazać, że obie korzystają z tego samego słownika ustawień.

Minimalne wymagania programu:

- klasa ConfigManager zaimplementowana jako Singleton,
- możliwość dodania ustawienia przez metodę `set_setting(key, value)`,
- możliwość odczytu ustawienia przez metodę `get_setting(key)`,
- test w programie głównym pokazujący, że `config1` i `config2` zwraca `True`.

9. Podsumowanie

Singleton rozwiązuje problem kontrolowanego tworzenia obiektów wtedy, gdy w programie potrzebna jest tylko jedna instancja danej klasy. W przykładzie z menedżerem konfiguracji wszystkie części programu korzystają z jednego wspólnego obiektu, dzięki czemu ustawienia pozostają spójne.